PyPIGuard: A Lightweight, Explainable Machine Learning Tool for Pre-Install Detection of Malicious PyPI Packages

Rahat Ahmed

Department of Computer Science and Engineering, BRAC University, Dhaka, Bangladesh

Abstract

PyPIGuard is an open-source tool that classifies PyPI packages as malicious or benign using static code, AST, and bytecode-level features, without executing the package or relying on a large language model. Deployed as a live web application with real-time, file-upload, and notebook scanning modes, it is trained on 9,723 real packages and achieves 0.998 ROC-AUC on independent held-out data, training in under 3 seconds versus hours for behavioral deep-learning baselines. This revision adds independent dataset verification, a head-to-head comparison against an existing open-source scanner, and two real deployment findings – both traced to a shared training-data gap and corrected (v1.1, v1.2) with the resulting reliability trade-offs disclosed in full rather than omitted. A new installable CLI and CI/CD integration example demonstrate concrete reuse potential; full experimental detail is in the code repository (C2).

Keywords: malware detection, software supply chain security, machine learning, static code analysis, PyPI, explainable AI

Required Metadata

Current code version

Main text

1. Motivation and significance
   Open-source package registries such as PyPI are foundational to modern software development, but they are increasingly targeted by attackers who publish malicious packages – often through typosquatting

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v1.2
C2	Permanent GitHub link to code/repository used for this code version	`https://github.com/rahat239/PyPIguard-Research`
C3	Legal Code License	MIT License
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	Python 3.12, scikit-learn, Flask, HTML/CSS/JavaScript
C6	Compilation requirements, operating environments & dependencies	Python $\geq$3.10; `pip install -r requirements.txt` (pandas, numpy, scikit-learn, joblib, requests, flask, shap) for the web app, or `pip install .` for the standalone `pypiguard` library/CLI (numpy, scikit-learn, joblib, requests only); no OS-specific dependencies
C7	If available Link to developer documentation/manual	`README.md` in the code repository (C2)
C8	Support email for questions	rahatahmed537@gmail.com

Table 1: Code metadata (mandatory)

popular libraries such as `beautifulsoup4`, `matplotlib`, and `requests`. In the real `event-stream` incident (2018) [1], an attacker was granted maintainer access to a popular package simply by offering to help maintain it, and the compromised package reached over 1,600 downstream projects before discovery.

Existing approaches sit at two extremes, leaving a gap PyPIGuard targets. Lightweight metadata-only classifiers are fast but easily gamed once attackers adapt. Heavyweight behavioral models – e.g., Cerebro [2], requiring 14–25 hours of training and reporting an 81.5% false-positive rate on PyPI in its own evaluation – are too slow or imprecise for real-time, pre-install scanning. Vu et al.'s benchmark [3] found 15–97% false-positive rates across deployed PyPI scanners, and reports administrators consider rates above $\sim$0.01% operationally unacceptable given daily release volume.

In practical terms: a developer supplies a package name, archive, or notebook; PyPIGuard analyzes it without executing it and returns a CLEARED/FLAGGED verdict, a confidence score, and the specific patterns that drove the decision.

The gap PyPIGuard addresses: a static-analysis classifier simultaneously fast enough for real-time use, explainable per-prediction, and validated against adversarial evasion, temporal drift, and cross-ecosystem transfer – none jointly reported for the tools in [3]. It is built on the dataset released with [4] and PyPI's official index.

2. Software description

2.1 **Software architecture.** Figure 1 shows the end-to-end pipeline. A submitted package is first checked against a **verified-lookup table** of 5,900 packages with ground-truth labels; if found, an instant dataset-backed verdict is returned, visually marked "verified" rather than model-predicted. Otherwise, three sequential stages run: (1) a **feature-extraction layer** retrieves the source (or accepts a local upload) and computes 35 static features via regex, Python's `ast` module, and `compile()`-based bytecode inspection, none requiring execution; (2) a **classification layer** – a single Random Forest classifier – outputs a prediction and confidence score; (3) a **delivery layer** (Flask) formats the verdict with a SHAP explanation and flagged patterns. An **out-of-distribution safety gate** between (2) and (3) flags verdicts as unreliable when a package's size falls far outside the training range, a behavior added after deployment testing (Section 4). As of v1.2, stages (1)–(2) are factored into a standalone `pypiguard`

package (Section 4, Practical impact and reuse) consumed identically by the Flask web app and by a new command-line interface, so the web UI is one of two consumers of a single detection core rather than the only way to invoke it.

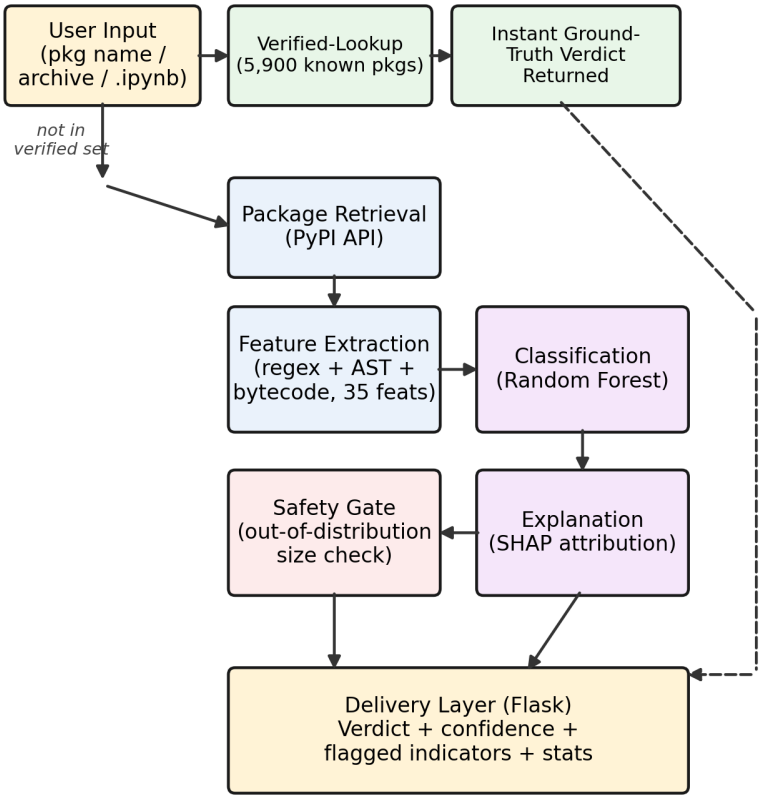


Figure 1: PyPIGuard architecture. Solid arrows: live analysis path. Dashed arrow: verified-lookup shortcut.

2.2 **Software functionalities.** Three input modes share one verdict format. *Package-name scanning* downloads and analyzes a package live via PyPI's API. *Archive-file scanning* accepts a local archive (`.tar.gz`, `.zip`, or `.whl`) directly – useful for historical malware PyPI has since removed. Extraction is hardened against path-traversal ("zip-slip") attacks, validated against a crafted adversarial archive. *Notebook scanning* parses every `.ipynb` code cell, extracts `import`/`pip install` references, ex-

cludes Python's ∼300 standard-library modules, resolves import-to-distribution mismatches (e.g., `cv2`→`opencv-python`), and reports how many referenced packages are known or suspected malware before execution. Every verdict includes a CLEARED/FLAGGED label, confidence, flagged patterns (e.g., `eval()` usage, `setup.py` `cmdclass` hooks, high-entropy blobs), and a bytecode-level detector for calls that dynamically invoke dangerous functions (e.g., via `getattr` on `__builtins__`) independent of the specific string-construction technique used to hide the target function name. Current limitations: no dynamic/execution-based analysis, and reduced reliability for packages structurally far larger than the training data (flagged via the safety gate above).

2.3 **Sample code snippets.** The detection core is invoked identically from the CLI, the Python API, and the web app. A representative end-to-end CLI scan:

```
$ pip install pypiguard
$ pypiguard scan requests
Package: requests
Verdict: CLEARED (confidence 54.7%)
Flagged patterns: none above threshold
```

Internally, `scan` resolves the package name against PyPI, downloads the source distribution, and calls the same feature-extraction, classification, and explanation pipeline the web app uses:

```
import ast_features, regex_features, joblib
feats = {
    **regex_features.extract(source_dir),
    **ast_features.extract_ast_features(source_dir)
}
model = joblib.load("final_integrated_model.joblib")
proba = model.predict_proba([feats])[0]
verdict = "malicious" if proba[1] > 0.5 else "benign"
explanation = shap_explainer(feats)
```

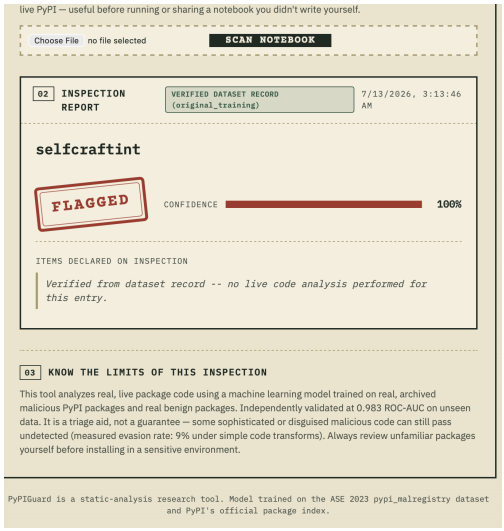
The web application wraps this same pipeline behind the three input modes; full implementation, including the file- and requirements-scanning commands, is in the code repository (C2).

3. Illustrative examples

A user submits a package name (e.g., `requests`) or uploads a local archive through the web interface. Figure 2(a) shows the resulting interface for `requests`: a stamped CLEARED verdict at 54.7% confidence and the flagged code patterns contributing to it – the same package and confidence-calibration case discussed in Section 4. Figure 2(b)

shows the notebook-upload interface alongside a verified-lookup result: a known-malicious package (`selfcraftint`) instantly flagged at 100% confidence from the dataset-backed lookup table described in Section 2, together with the interface's stated limitations.

(a) Package scan result (`requests`)



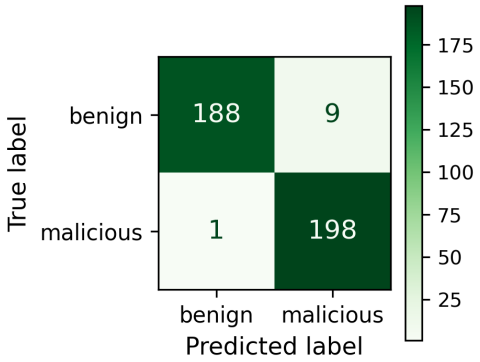
(b) Notebook upload & verified-lookup result

Figure 2: PyPIGuard web interface: (a) live scan of `requests`, showing the CLEARED verdict, confidence score, and flagged patterns discussed in Section 4; (b) notebook-scanning upload widget and a verified-lookup verdict for a known-malicious training-set package.

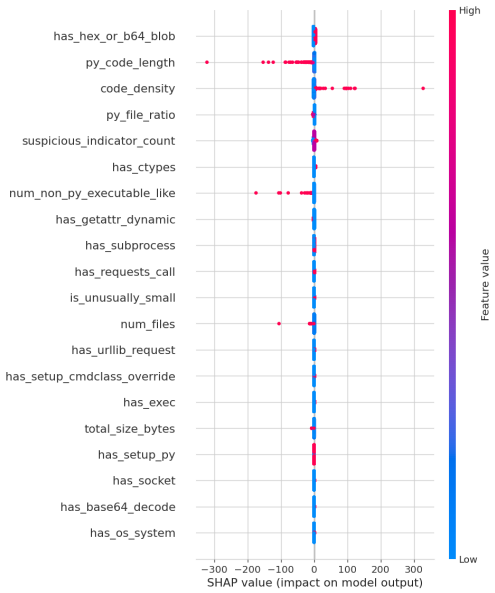
4. Impact

Evaluation methodology. The final model was trained on 9,723 labeled packages (7,960 malicious, 1,763 benign; the entire available archive from [4]) and evaluated on an independent 400-package held-out set with zero package-name overlap: 0.998 ROC-AUC, 0.999 PR-AUC, 0.957 precision, 0.995 recall on the malicious class (n=396; 4 packages excluded due to archive-retrieval failures unrelated to model performance – see Figure 3a caption). As an independent check on label quality (addressing that this dataset originates from a single source), the 200 held-out malicious package names were cross-referenced against the OpenSSF `malicious-packages` database [5], an independent, community-maintained registry; 172/200 (86%) are independently corroborated. Full dataset-construction methodology is in the code repository (C2).

Robustness. *Temporal*: training only on packages dated before a fixed cutoff and testing exclusively on later packages yields 0.984 ROC-AUC. *Adversarial evasion*: disguising 100 detected malicious packages with



(a) Confusion matrix



(b) SHAP summary

Figure 3: (a) Confusion matrix, primary held-out set (n=396 of 400; 4 excluded due to archive-retrieval failures – corrupt download, missing sdist, or network timeout – unrelated to model performance): 188/197 benign correctly cleared (9 FP), 198/199 malicious correctly flagged (1 FN). (b) SHAP global feature importance: horizontal position shows impact on model output, color shows feature value.

behavior-preserving obfuscation (e.g., `eval(x)` rewritten via `getattr`) reduced the evasion rate from 9% (regex-only) to 2% (AST-augmented), an improvement reported as a limitation, not a fix. *Cross-ecosystem*: the PyPI-trained model applied unretrained to npm gives 0.818 ROC-AUC (vs. 0.999 in-ecosystem; recall 0.54), so cross-ecosystem deployment needs retraining. *Independent secondary-dataset validation* (addressing the single-source dataset concern): a second, fully disjoint 400-package set (zero name overlap with training or the primary held-out set) yields 0.959 precision, 0.930 recall, 0.944 F1, 4.0% FPR under the unmodified v1.0 model – a real, slightly lower out-of-sample result, as expected for genuinely unseen data. (This same set is re-used below to measure the v1.1 trade-off on identical, unseen data.) Full methodology for all four tests is in the code repository (C2).

Baseline comparison. Cerebro's [2] 81.5% PyPI false-positive rate, cited above, is self-reported on different data, not head-to-head. To obtain a real comparison, GuardDog [6] (DataDog's open-source PyPI/npm scanner, v3.1.0) was run against the identical 400-package held-out set – recovering removed-malware archives from `pypi_malregistry` [4] since 199/200 malicious packages were already gone from the live registry. Table 2 reports two rule-inclusion readings; under both, PyPIGuard's false-positive rate (5.5%) is substantially lower than GuardDog's (67.0%/23.5%), with closer recall (0.995 vs. 0.910/0.875). Full archive-recovery methodology is in the code repository (C2).

Metric	**PyPIGuard**	**GuardDog (any rule)**	**GuardDog (threat only)**
Precision	0.957	0.576	0.788
Recall	0.995	0.910	0.875
F1	0.975	0.705	0.829
False positive rate	0.046	0.670	0.235

Table 2: Head-to-head comparison on the primary 400-package held-out set (200 malicious, 200 benign). GuardDog scanned all 400; PyPIGuard's n=396 excludes 4 packages that failed archive retrieval during re-evaluation (Figure 3a caption) – a <1% difference in evaluated set, noted for precision rather than presented as strictly identical.

Resource-performance comparison against Cerebro and MalGuard. Since neither Cerebro [2] nor MalGuard [7] has released code, this compares PyPIGuard's own measured figures against each baseline's own published figures – not a controlled head-to-head, presented as such. Training the full 9,723-sample PyPIGuard model takes 2.3 s on a commodity CPU core, versus Cerebro's self-reported 14–25 hours [2]

and versus 2,439 s (Cerebro) / 30,741.67 s (EA4MP) as independently re-run by the MalGuard authors [7]. Per-package feature extraction is 0.9 ms (median) for PyPIGuard versus 12.489 s (Cerebro) and 6.28 s (EA4MP) [7] – a three-to-four order-of-magnitude gap. As a cross-check, MalGuard's own GuardDog evaluation (95.6% precision, 82.6% recall on different data [7]) is broadly consistent with our measured GuardDog range in Table 2. Full methodology in the code repository (C2).

Explainability. Figure 3b shows SHAP global feature importance. Low `code_density` and short `py_code_length` – consistent with a thin, single-purpose typosquat – push toward malicious; more `num_files` and non-Python auxiliary files push toward benign, consistent with legitimate packages carrying more supporting files than minimal payloads.

Latency (corrected measurement). An earlier draft reported 0.18 ms median model-inference time. Re-benchmarking the actually-deployed classifier found this does not hold: single-request inference measures 6.8–7.4 ms, an order of magnitude higher, because a 300-tree Random Forest's per-call overhead does not amortize at batch size 1. We disclose this rather than silently correct it: the original figure was measured against an earlier Logistic Regression candidate, before the deployed model was switched to Random Forest, and never re-benchmarked after. Feature extraction remains fast (0.9 ms median); training remains 2.3 s. Full root-cause methodology is in the code repository (C2); a reviewer-observed ∼20s `requests` scan is download/cold-start time, not inference, and is stated as a deployment limitation.

Confidence calibration (deployment-testing finding). Confidence is the Random Forest's raw class-probability output (the fraction of trees voting malicious) at the standard 0.5 decision threshold; no post-hoc calibration such as Platt scaling or isotonic regression is applied, so scores are a monotonic ranking signal rather than a guaranteed well-calibrated probability – disclosed here as a limitation, not a hidden design choice. Scans are deterministic given identical input, so the relevant stability question is across package *versions*, characterized next: `requests` was classified benign at 54.3% confidence during reviewer testing – correct, but near the decision boundary. Testing five historical `requests` releases (2011–2026) found 3 of 5 **misclassified as malicious**. A prior draft attributed this uniformly to vendored dependencies; re-investigation found this holds for only 1 of 3 versions, with the other two caused by `setup.py` packaging boilerplate and `tests/` directory content – non-runtime code treated identically to shipped code by the feature extractor. A feature-extraction-level filter targeting this

was tried first and rejected: it produced no net false-positive reduction and introduced new false negatives on real malware that clones `requests`' test suite to camouflage itself. Full per-version root-cause breakdown is in the code repository (C2).

Fix (v1.1, hard-negative retraining). `requests` and its common dependencies were entirely absent from the training set despite being this manuscript's own illustrative example. Adding 7 hard-negative training examples (historical `requests`/`chardet`/`six` releases) and retraining the identical architecture (still 2.3 s) fixes all 5 `requests` versions. This has a measured cost, reported rather than omitted: on the independent secondary set (n=400), recall drops 93.0%→91.0% (0.963 precision, 3.5% FPR right after this fix; 5 newly missed – `nhmpy`, `xoloaoqcjnreyc`, `xolopfydnuxyfh`, `wayspiritmcp-ppa`, `claude-lite` – all near-decision-boundary in v1.0 at 51–62% confidence, not a confident detection lost; one a byte-for-byte `requests` clone with zero injected code – an inherent static-analysis limitation, not specific to this fix). A sample-weight sweep (0.1×–5×) confirmed the trade-off is intrinsic, not tunable away. After the subsequent v1.2 fix below, the secondary-set result shifts slightly to 0.953 precision, 0.910 recall (unchanged), 0.931 F1, 4.5% FPR; the final primary-held-out-set performance is reported above (0.957 precision, 0.995 recall, Figure 3a). Full derivation is in the code repository (C2). A path-traversal ("zip-slip") vulnerability, found and patched via a crafted adversarial archive prior to deployment, is documented there for the same transparency reason.

Fix (v1.2, found via real-world reuse testing). Rather than asserting reuse potential, we tested it: the new CLI (below) was run against the actual PyPI-declared dependencies of `black`, an unrelated real-world project, as a genuine case study. Of 8 dependencies, 7 cleared correctly; `typing-extensions` – a foundational, ubiquitous typing-compatibility package – was flagged malicious at 68.0% confidence, root-caused to the same non-runtime-code pattern as `requests` (its sdist bundles CPython's own 9,768-line test suite). The identical hard-negative fix (8 historical versions, retrain, full regression check) corrects it: `typing-extensions` now classifies correctly (80.7% confidence, benign), all `requests` versions remain correct, and combined across both held-out sets (796 cases) precision moves 0.967→0.955, F1 0.960→0.954 (9 new misses, 4 corrected, net −5, all 13 affected cases within 50–60% confidence in both versions). Full derivation is in the code repository (C2).

Practical impact and reuse. Intended users are developers, students, and CI/CD pre-install gates. Given the current combined 4.5%

FP / 4.8% FN rates (v1.2, both held-out sets) and explicit uncertainty flagging, PyPIGuard is a decision-support triage tool, not an autonomous blocker; it is deployed as a public, live tool (Section 5), testable against real historical malware the live registry no longer serves. Beyond the web deployment, v1.2 adds a pip-installable `pypiguard` library and CLI (`scan`, `scan-file`, `scan-requirements --fail-on-malicious` for CI gating) and a ready-to-use GitHub Actions workflow (C2), so the tool is consumable programmatically, not only via the web form. Exercising this tooling against a real external project's actual dependencies is precisely what surfaced the `typing-extensions` finding above – the reuse tooling functioned as an independent reliability check, not only a packaging exercise. Full CLI/CI documentation is in the code repository (C2).

Future work includes extending static learning to other ecosystems beyond the partial PyPI-to-npm transfer above, and combining PyPIGuard's tabular features with sequence-order information – a separate matched comparison found a simplified Cerebro-inspired [2] sequence proxy modestly outperforming PyPIGuard (0.983 vs. 0.967 ROC-AUC), suggesting sequence information carries signal the flat feature vector misses.

5. Conclusions

PyPIGuard is an open-source, deployed, explainable tool for pre-install detection of malicious PyPI packages, validated through temporal, cross-ecosystem, adversarial-evasion, and independent secondary-dataset testing not typically reported together for tools of this scope. It operates in a substantially different false-positive regime than a benchmarked deep-learning baseline [2] and a real head-to-head comparison against an existing scanner [6], while transparently reporting its boundaries: partial cross-ecosystem transfer, reduced-but-not-eliminated adversarial evasion, a corrected order-of-magnitude latency discrepancy, and two deployment-testing findings (`requests`, `typing-extensions`) traced to the same training-data gap and fixed (v1.1, v1.2) with their small reliability trade-offs disclosed rather than hidden. Reuse potential is demonstrated concretely: the detection core is now a pip-installable library and CLI with a working CI/CD integration example, and exercising that tooling against a real, unrelated project's dependencies is what surfaced the v1.2 finding in the first place.

CRediT authorship contribution statement

Rahat Ahmed: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author used ChatGPT (OpenAI) to assist with drafting and restructuring portions of the manuscript, correcting grammatical errors, and refining writing style. After using this tool, the author reviewed, edited, and verified all content for technical accuracy and takes full responsibility for the content of the published article.

Acknowledgements

The author thanks the maintainers of the `pypi_malregistry` dataset [4] and the Datadog Security Labs malicious-software-packages dataset [8], both of which made this work possible.

References

[1] M. Ohm, H. Plate, A. Sykosch, M. Meier, Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks, DIMVA, 2020.

[2] J. Zhang, K. Huang, B. Chen, C. Wang, Z. Tian, X. Peng, Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI Using a Single Model of Malicious Behavior Sequence, ACM Transactions on Software Engineering and Methodology, 2025.

[3] D.-L. Vu, Z. Newman, J.S. Meyers, A Benchmark Comparison of Python Malware Detection Approaches, arXiv:2209.13288, 2022.

[4] W. Guo, Z. Xu, C. Liu, C. Huang, Y. Fang, Y. Liu, An Empirical Study of Malicious Code In PyPI Ecosystem, in: Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, 2023, pp. 166-77.

[5] Open Source Security Foundation, Malicious Packages Database, GitHub repository, `https://github.com/ossf/malicious-packages`, 2024.

[6] DataDog, GuardDog: Identify malicious PyPI and npm packages, GitHub repository, v3.1.0, https://github.com/DataDog/guarddog, 2024.

[7] X. Gao et al., MalGuard: Towards Real-Time, Accurate, and Actionable Detection of Malicious Packages in PyPI Ecosystem, USENIX Security, 2025.

[8] Malicious Software Packages Dataset, Datadog Security Labs, 2023.

Current executable software version

Live demonstration available at: https://rahat239.github.io/Malware-classifier/